# Rapid Conversational Authoring of Interactive Human-Robot Activities

**Shiye Cao**[1*], **Aureliane Tchamdja**[2*], **Chien-Ming Huang**[1]

[1]Johns Hopkins University
[2]The University of Maryland, Baltimore County
shiyecao@cs.jhu.edu, mtchamd1@umbc.edu, cmhuang@cs.jhu.edu

## Abstract

Social robots are being envisioned for a growing range of applications. Yet their behavior is still predominantly authored by robot experts and developers through hand-crafted interaction content or, more recently, large language model (LLM) prompts. This places control over robot behavior with those who implement the interaction rather than the domain experts and end users who better understand how the robot should serve users' evolving needs over time. Existing end-user authoring tools, such as visual programming environments, lower this barrier but still require specialized authoring skills, including constructing interaction logic through blocks and flows. We present the Robot Interaction Development Environment (RIDE), an open-source, end-to-end, mobile-compatible system that automates much of the human-robot interaction authoring process. From a natural-language description of an activity, RIDE decomposes the interaction into a sequence of states, generates the interaction flow and state-level prompts that drive the robot, and deploys the resulting program. We evaluated RIDE through three case studies in which a speech language therapist designed social-skills practice for a patient, a high school French teacher designed conversation practice for a student, and a nurse designed a stress-relief activity for herself. All three participants created a deployable activity within seven conversational turns without writing prompts or program logic. While participants readily delegated interaction structure to the system, they retained control over domain-specific details such as target phrases and content to avoid. We discuss how low-effort, conversational authoring can enable domain experts and end users to rapidly adapt general-purpose social robots to their goals and evolving needs.

## Introduction

As social robots move beyond laboratory settings, they are being explored for therapy, education, coaching, and companionship. Their usefulness in these settings depends heavily on the content and structure of the interaction: what the robot says, what it asks, and how the activity is organized around a particular user and goal. Today, this content is still predominantly authored by robot experts and developers, either through hand-crafted dialogue and behavior scripts or, more recently, through large language model (LLM) prompts (Csapo et al. 2012; Cao et al. 2025). Both approaches require expertise that many stakeholders in these settings do not have (Cao et al. 2026; Matheus et al. 2025).

Domain experts and end users, however, hold much of the knowledge needed to determine how these interactions should unfold. Teachers and therapists understand the goals of an activity, strategies that work for a particular person, and content that should be included or avoided. End users understand their own preferences, comfort, and pace. This knowledge can be highly specific and consequential, such as a target phrase a student is practicing or a topic a patient finds distressing. Yet incorporating it into robot behavior often requires mediation by a developer. This becomes increasingly difficult to sustain over long-term robot use, where authoring is recurring rather than one-time and interaction needs evolve as skills develop, interests shift, and difficulty is adjusted (Jain et al. 2020).

End-user authoring systems aim to reduce this gap. Visual and block-based programming environments allow novices to assemble robot behaviors without writing code syntax (Weintrop et al. 2018; Coronado et al. 2020). Although these systems lower implementation barriers, users must still decompose an intended interaction into steps, sequence those steps, and reason about branching and state. Recent work has used generative AI to scaffold this process by helping users translate interaction goals into programs, often with the additional aim of supporting the development of programming skills (Sato et al. 2026). However, users may still need to understand and manipulate the underlying program representation, and generated suggestions may not always correspond to capabilities available in the interface. Many existing systems also assume desktop-based, dedicated authoring sessions, which may not fit the brief opportunities domain experts have to prepare or adapt an interaction in practice. Therefore, we pursue a different design target by asking how much of the implementation can be automated by the system, how little users need to contribute before a runnable interaction exists, and whether the remaining authoring effort is lightweight enough to be completed on a mobile device.

We present the Robot Interaction Development Environment (RIDE), an open-source, end-to-end, mobile-compatible system that automates much of the human-robot interaction authoring process. Users describe the activity they want through text or voice, and RIDE decomposes the in-

*These authors contributed equally.

Copyright © 2027, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

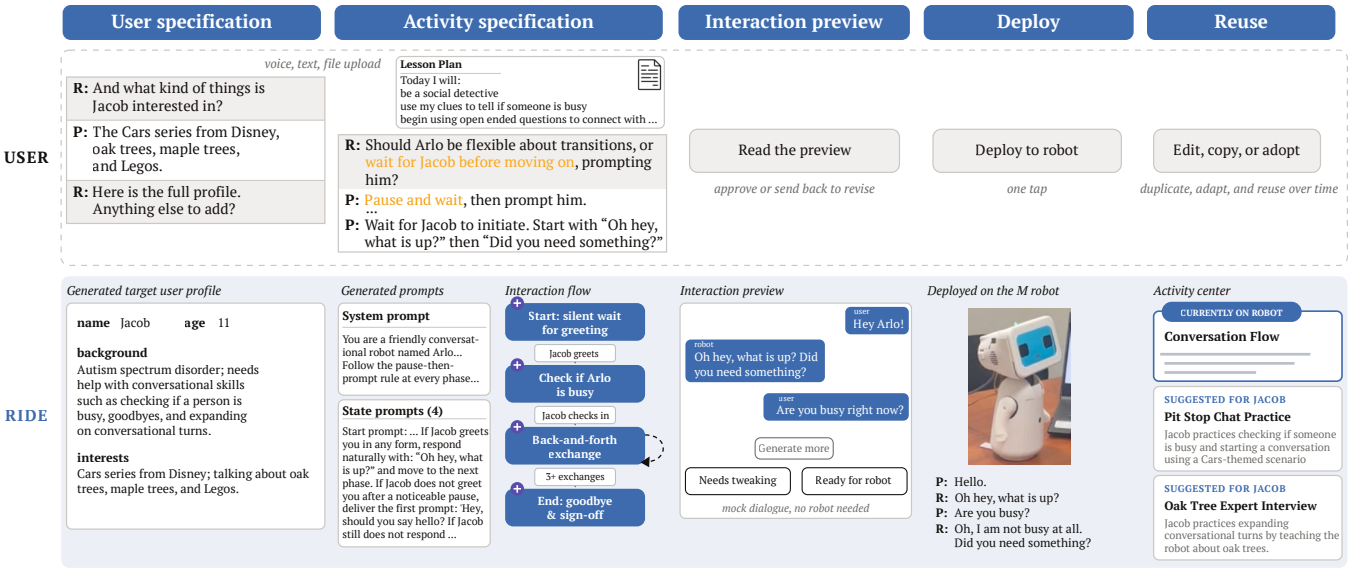



Figure 1: We present the Robot Interaction Development Environment (RIDE), an open-source, end-to-end, mobile-compatible system that automates much of the human-robot interaction authoring process. Example shows P1, a speech language therapist with no programming experience, authoring an activity for one of her patients to practice social conversational skills. Recommendations P1 accepted highlighted in orange. "R" denotes RIDE voice agent. "P" denotes participant.

teraction into states, generates the interaction flow and state-level prompts that drive the robot, previews the resulting interaction, and deploys the program to the robot. RIDE is organized into four modules (see Fig. 1): target user specification, activity specification, interaction preview, and activity center. More specifically, RIDE is designed with three capabilities aimed at reducing the cost of authoring:

- AI-scaffolded elicitation that incorporates reference materials, prompts users about key interaction elements, and proposes candidate specifications, reducing the effort of translating domain knowledge into interaction logic.
- Automated task decomposition, program assembly, and prompt engineering from a natural-language activity description, allowing users to focus on what the interaction should be rather than how to implement it.
- Interaction preview that simulates an exchange from the current design, allowing users to review and validate an activity before committing to a live robot session.

We evaluated RIDE through three case studies with domain experts and end users authoring interactions for their own contexts: a speech language therapist designing social-skills practice for a patient, a high school French teacher designing conversation practice for her student, and a nurse designing a stress-relief activity for herself. All three participants created a deployable activity within seven conversational turns without writing prompts or program logic. Participants delegated creating high-level interaction structure to RIDE while retaining control over domain-specific details, such as target phrases and content to avoid. They also tolerated minor interaction glitches when these did not disrupt the activity, suggesting that low-effort authoring may prioritize coherent, goal-aligned interactions over fine-grained control of every behavior.

Overall, our work makes the following contributions:

- The design, implementation, and open-sourcing of RIDE[1].
- Three case studies demonstrating the initial feasibility and usability of RIDE for authoring personalized human-robot activities.
- Design implications for conversational authoring systems that support rapid adaptation of general-purpose social robots to evolving user needs.

## Related Work

### End-User Robot Programming Systems

End-user robot programming aims to enable people without programming or robotics expertise to author robot behavior (Ajaykumar, Steele, and Huang 2021). Prior work has explored non-programming approaches such as programming by demonstration, in which users physically guide a robot or demonstrate a task (Akgun et al. 2012); embodied and tangible approaches such as bodystorming and tabletop authoring (Porfirio et al. 2019, 2021; Sefidgar et al. 2018); and augmented or virtual reality interfaces for programming behavior in relation to the physical environment (Gao and Huang 2019; Kapinus et al. 2019; Ikeda and Szafir 2024; Yigitbas, Jovanovikj, and Engels 2021).

Visual programming environments have also been widely adopted because they are more accessible to novices than traditional text-based programming languages (Coronado et al. 2020; Weintrop and Wilensky 2017). HRI authoring tools have used data-flow interfaces (Alexandrova, Tatlock, and

[1]RIDE code available on GitHub.

Cakmak 2015; Pot et al. 2009), behavior trees (Paxton et al. 2017), and block-based programming (Weintrop et al. 2017; Sato et al. 2026; Schoen and Mutlu 2024). Other systems embed robotics knowledge to guide or correct user programs (Schoen et al. 2022; Sanders, Gegov, and Ndzi 2016), or use familiar interfaces such as structured English (Buchina, Kamel, and Barakova 2016) and spreadsheets with natural-language instructions (Swaminathan et al. 2024).

These approaches reduce implementation barriers but still require users to decide what an interaction should contain, decompose it into steps, sequence those steps, reason about branching and state, and may still present a learning curve for users with no prior programming experience, especially as the interaction imagined is complex (Swaminathan et al. 2024; Ajaykumar, Steele, and Huang 2021; Sato et al. 2026). For example, users of a spreadsheet-based interface found writing instructions for individual blocks difficult and were limited to short programs without loops (Swaminathan et al. 2024). In contrast, our tool (RIDE) uses a generative AI-powered voice agent to scaffold interaction design and automates decomposition, sequencing, branching, prompt construction, programming, and testing, further reducing the amount of effort required from users to create and revise interactions.

### LLMs for Social Robot Interaction Authoring

Large language models have increasingly been used to generate social robot behavior, including conversation content (Cao et al. 2026), facial expressions (Antony, Stiber, and Huang 2025), and stories paired with expressive delivery (Antony et al. 2026a). These approaches expand the range and adaptability of robot interactions by allowing behavior to be generated from natural-language specifications rather than authored entirely by hand.

However, configuring LLM-powered interactions introduces new authoring challenges. Prompt engineering is not intuitive and requires practice (Oppenlaender, Linder, and Silvennoinen 2025), while non-experts often rely on trial and error, have incomplete mental models of LLM capabilities, and are uncertain how to translate their goals into effective prompts (Zamfirescu-Pereira et al. 2023). Prior work has begun to scaffold this process. Voice-based LLM agents can help users construct an initial social robot behavior prompt and translate feedback from observed interactions into prompt revisions (Cao et al. 2026). Generative AI has also been integrated with visual robot programming environments to help users construct and refine social robot behavior. However, the goal of the system was targeted at helping users develop programming skills and reduce overreliance on the tool (Sato et al. 2026).

Our tool builds on this work and prior work from LLM-powered agent design (Wu et al. 2022) but shifts the focus from helping users author prompts or programs to the aim of minimizing the need for users to engage directly in prompt engineering or interaction programming when creating personalized social robot activities through automation. Our goal is to make authoring lightweight enough for domain experts and end users to create and revise robot activities on the fly as needed. RIDE therefore focuses on supporting rapid creation from a short description, reuse and duplication of prior activities, conversational revision, and activity suggestions grounded in a target user's profile. In doing so, we aim to make personalized human-robot interaction practical to author repeatedly over long-term robot use.

## Robot Interaction Development Environment

The Robot Interaction Development Environment (RIDE) is an end-to-end, mobile-compatible authoring system for social robots. RIDE uses large language models (LLMs) to let users create human-robot interactions through natural language rather than programming. The system supports four main modules (see Fig. 1): target user specification, activity specification, interaction preview, and activity center.

A separate robot configuration page manages robot-level settings shared across activities. It allows users to provide a name for the robot, choose a voice, specify instructions for the voice, specify up to two languages for the robot, and select robot facial expressions and body movements from a bank of options.

The activity specification and document-processing agents use Claude Sonnet 4.6, while the target user specification agent uses GPT-4o-mini to reduce response latency. RIDE voice agent speech is synthesized using OpenAI's tts-1 model, and user speech is transcribed using Google Cloud Speech-to-Text.

### Target User Specification

This module captures information about the intended user or user group so that activities can be tailored accordingly. Users may enter this information manually or provide it conversationally through text or speech. The RIDE agent extracts relevant details and automatically fills fields for the target user's name, age, skill level and background, and interests while preserving previously confirmed values.

Users may also upload supporting materials relevant to the target user, including PDFs, images, and Word documents. A document-processing agent extracts and summarizes their contents and uses the resulting information to populate the same profile fields.

### Activity Specification

This module supports conversational activity design (see Fig. 2). Rather than asking users to provide a complete activity specification, the RIDE agent begins by asking them to describe the scenario they have in mind. This reduces the blank-page problem commonly observed in prompt and LLM-agent authoring (Cao et al. 2026; Zamfirescu-Pereira et al. 2023; Zhang and Arawjo 2025). The agent then proposes candidate specifications and asks clarifying questions about the activity's goals, content, and desired robot behavior (see example in Fig. 4), helping users refine the interaction without requiring them to formulate a complete specification upfront. Users may also upload reference materials, such as PDFs, images, or Word documents, during activity creation. A document-processing agent extracts and summarizes their contents for use by the RIDE agent during specification. Content that

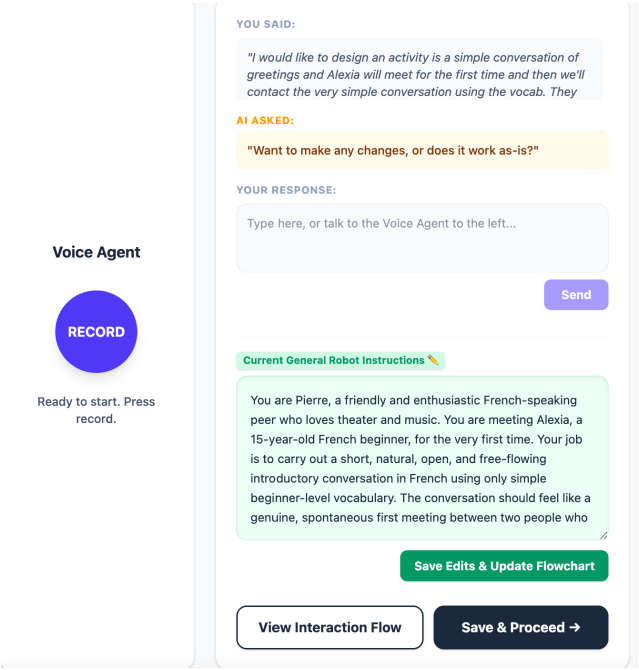



Figure 2: The activity specification page for P2 from the RIDE interface.

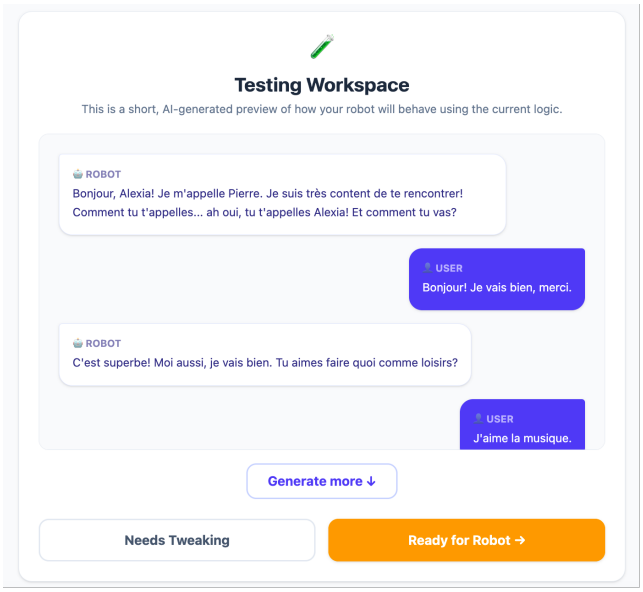



Figure 3: The interaction preview page for P2 from the RIDE interface.

may be important to preserve, such as vocabulary terms, is retained in the summary.

RIDE translates the resulting natural-language specification into a graph of state-level prompts. This representation combines the activity-flow structure common in visual robot programming with prompt chaining used in LLM applications. Prior work has shown that decomposing complex tasks into multiple prompt-driven steps can improve agent performance on complex tasks, transparency, and controllability, but it requires users to define both the task decomposition and the prompts for individual steps (Wu et al. 2022; Wu, Terry, and Cai 2022). RIDE automates both.

Generation proceeds in three stages. First, the RIDE agent produces an overarching activity prompt that captures the activity goals, behavioral guidelines, and intended flow. Second, it decomposes the activity into states (nodes) and transitions (edges) to form an activity-flow graph (see example in Fig. 4). Third, it generates a state-level behavior prompt for each node, specifying how the robot should behave during that portion of the interaction.

All generated content remains available for inspection and editing. The overarching activity prompt is displayed directly in the interface, while state-level prompts can be accessed by selecting nodes in the activity-flow graph. The overarching prompt is used to generate the flow and state prompts rather than to control the robot directly. When users revise it, RIDE regenerates the activity flow and corresponding state-level prompts.

## Interaction Preview

This module allows users to inspect how an activity may unfold before deploying it to a physical robot (see Fig. 3). Following prior work on LLM-agent authoring (Zhang and Arawjo 2025), RIDE uses a mock-conversation agent to simulate an exchange based on the current activity design. Robot turns are generated from the activity prompts, while hypothetical user responses are generated using the target user profile. This allows users to identify potential misalignment before on-robot testing.

## Activity Center

The activity center serves as a library of previously created activities. Users can edit, duplicate, delete, and deploy stored activities, and the activity currently running on the robot is indicated with a badge. This allows users to easily duplicate, adapt, and reuse activities over time. Editing an activity reopens the conversational authoring interface with its existing activity prompt, flow graph, and conversation history. Duplicating an activity creates a new copy containing the same prompt, flow graph, conversation history, and attached reference materials, which can then be further customized. Unsaved activities are retained as drafts for the remainder of the login session and can be resumed from where the conversation ended. The activity center also presents three activity recommendations generated from the target user profile. Users can adopt a recommendation directly or use it as a starting point for further customization.

*Target User Profile*

**name**: [PSEUDONYM] **targetAge**: "60" **interests**: "Reducing stress" **cognitiveProfile**: "I'm a critical thinker, like to have quality work"

**R**: Press record and describe your scenario to start creating the activity. Please keep it short and simple — the app will help you refine it from there. Ex: "I want to create an activity where…"

**P**: Help

**R**: Welcome! It sounds like you want to build a custom activity — could you describe in a sentence or two what you have in mind? For example, is it a **calming guided exercise**, an open conversation about stress, a game, or something else entirely?

**P: Calming guided exercise**

**R**: Great choice! To help me shape this further — what kind of calming exercise did you have in mind? For example, is it a breathing exercise, a body scan, a guided visualization, a mindfulness check-in, or a **combination of these**?

**P**: **Combination of these**

**R**: Wonderful. Now, how does the robot know when to move from one part of the exercise to the next — for example, does the user say something like **'ready' or 'done,'** do you want the robot to move on after a set amount of time, or does the robot check in with a question and wait for the user's response?

**P**: **Ready or done**

**R**: That is clear and simple — perfect for a calming experience. Now, how should the robot handle it if the user seems distressed or says something like they cannot relax or this is not working — should the robot **offer encouragement** and continue, pause and offer an alternative, or something else?

**P: Offer encouragement**

**R**: Lovely. Now let us think about how the exercise ends — what condition should bring the session to a close? For example, does it end after all four parts are completed, or should the **robot check in** at the end and ask if the user would like to repeat any part or simply close with a farewell?

**P**: **Robot to check in**

**R**: This is shaping up beautifully. One last question — how would you like the robot to open the session? For example, should it greet the user, briefly explain what the exercise involves, and ask if they are ready to begin, or would you prefer a different kind of opening?

**P**: All of the choices given

**R**: Your activity is fully designed and the prompt is ready to use. Just to confirm — is there anything you would like to adjust, such as the tone, the order of the exercise parts, the specific wording the robot uses, or anything else?

*Generated system prompt (excerpt)*

You are a warm, calm, conversational social robot guiding an adult through a multi-part calming exercise designed to reduce stress. Speak slowly and …

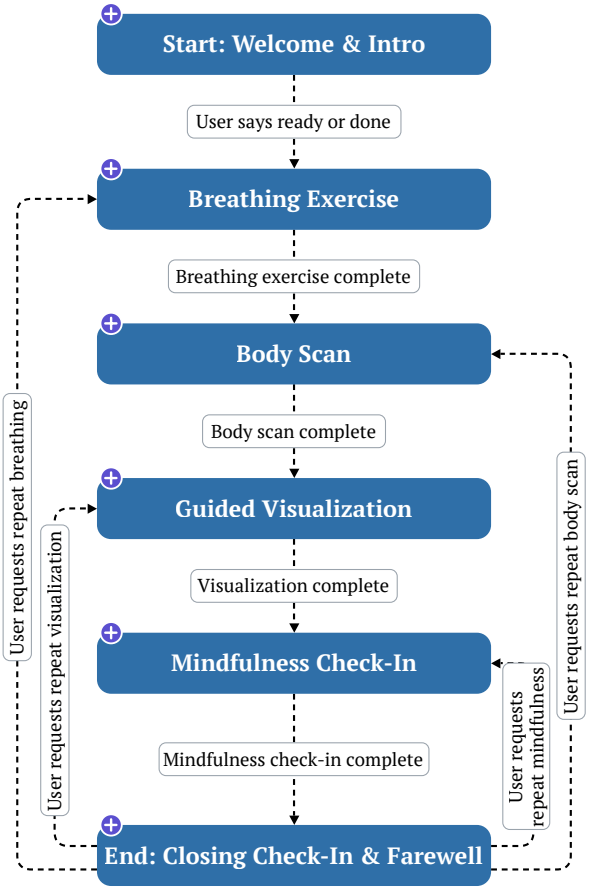
**OPENING:** Greet the user, explain the four practices, ask if they are ready, and wait for 'ready' or 'done.' …

**EACH PART:** Guide the user one step at a time, pausing after every instruction until they say 'ready' or 'done.' …

**CLOSING CHECK-IN:** Ask how they feel and whether to repeat a part or finish, then close with a warm farewell. …

**ENCOURAGEMENT:** If the user says they cannot relax, answer with warmth and invite them to continue at their own pace. …

*Generated interaction flow*




**Breathing Exercise** *expanded: generated state prompt (excerpt)*

**Task:** You are a calm, soothing social robot guiding an adult through a step-by-step breathing exercise — the first of four calming practices. Pace it gently, …

**Required content:** Guide a simple sequence: slow inhale, hold, exhale. Deliver one instruction, then wait for 'ready' or 'done' before the next. Repeat for three to four cycles. …

**Style:** Slow, steady, almost rhythmic. Minimal, calming language. …

**Output format:** Return a single JSON array. Every utterance carries an expression and a gesture with a timing marker, chosen only from the provided lists. Each utterance must be 10 words or fewer. …

**Interaction tags:** Wrap any question to the user in `<interaction role="breathing_exercise">`

**Transitions:** If the breathing exercise is complete, transition to state_3. If the user asks to stop, transition to state_6 with a warm goodbye. …

Figure 4: P3 specifying the stress-relief interaction in RIDE. Using the profile from target user specification (top) and information gathered from the activity specification conversation (left), RIDE generates an overall system prompt for the user to review (right), a state-based interaction flow graph, and state-by-state prompts for the robot (one example on the bottom). We highlight recommendations that P3 accepted from RIDE in orange. **R** denotes RIDE voice agent and **P** denotes P3.

## Case Studies

We conducted three case studies with two domain experts and one end user (see participant demographics in Table 1) to assess the feasibility and usability of RIDE, examine the resulting interactions, and identify limitations of the system.

### Procedure

After providing consent and completing a demographics survey, participants were introduced to the M robot and asked what kinds of social activities they could imagine the robot supporting in their work or personal life. Participants then watched a six-minute video walkthrough of the RIDE interface. They were asked which device they would naturally use to author robot activities in the scenario and completed the design task using their preferred device. After designing an activity, participants tested it on the robot and completed a post-study questionnaire about the authoring process, RIDE, and the resulting robot interaction. The study concluded with a semi-structured interview about their overall experience and feedback on the system. All sessions were audio- and video-recorded. The study was approved by our institutional review board (IRB), and participants were compensated at a rate of $15 per hour.

### Metrics

We measured participants' satisfaction with the resulting human-robot activity using a six-item Likert-scale questionnaire capturing satisfaction and confidence in the activity. We assessed system usability using the System Usability Scale (SUS), a 10-item, five-point Likert-scale questionnaire (Brooke 1996). SUS scores above 70 on the 0–100 scale are generally considered indicative of good usability (Bangor, Kortum, and Miller 2009). Finally, participants rated their overall enjoyment of designing the robot activity on a five-point Likert scale.

### Findings

Participants spent $24$ minutes with RIDE on average: $4$ minutes specifying the target user, $17$ minutes authoring the activity, and $3$ minutes previewing it. Note that this activity authoring time includes the time P1 and P2 also spent locating and uploading reference materials. All three participants completed activity specification within seven conversational turns with RIDE, using an average of $18.76$ words per turn. All three participants enjoyed the activity design process ($4.33/5$) and were generally satisfied with the resulting human-robot activity ($4.28/5$). The system usability score was $72.5/100$ on average. Next, we describe each case study in detail.

**Case Study 1:** P1 works as a speech language therapist. She designed a practice activity for Jacob, a patient with autism spectrum disorder who needed support with conversational skills, including checking whether someone is available, taking conversational turns, changing topics appropriately, and ending a conversation (see Fig. 1). Because the activity was intended for use during a therapy session, P1 began by locating her lesson plan and uploading it to RIDE as a reference document. She spent $37$ minutes designing the activity from scratch overall. She took longer than the other two participants because she spent extra time finding the lesson plan document. While she delegated the overall interaction structure to RIDE, she specified individual robot turns and contingencies when they mattered clinically. For example, when RIDE asked whether the robot should introduce one of Jacob's interests or wait for Jacob to bring it up, P1 chose the latter and added that, if Jacob continued discussing the topic for more than three conversational turns, the robot should say, "I don't want to talk about [topic] right now." These choices allowed P1 to preserve specific therapeutic goals while leaving the broader interaction structure to RIDE. The resulting activity guided Jacob through greeting someone, checking their availability, taking turns asking and answering questions, transitioning between topics, and saying goodbye, with gentle prompts when needed.

P1 reported high enjoyment of the design process ($5/5$) and rated RIDE's usability at $92.5$ on the SUS. She described the system as "very intuitive" and particularly valued RIDE's conversational scaffolding. She liked that RIDE kept asking for clarification, explaining that, "because I don't use AI often, I didn't know I needed to prompt it so specifically, but it was really looking for what I needed". She also valued the integrated voice input, which she felt could be a "huge time save" given the amount of prompting involved in the design process.

Overall, P1 felt that "[RIDE] was able to create the activity I had in mind" and was satisfied with the resulting activity ($4.33/5$). During on-robot testing, she was pleased that the robot followed her instruction not to over-engage with Jacob's specific topics of interest, a behavior she had explicitly requested during design. She also valued how the robot prompted Jacob through the exercise. However, she noticed that parts of the conversation did not flow naturally. She explained that "open-ended questions is how most of the world communicates" and noted that "we were so focused on [a specific topic], and that's not how a conversation flows." Despite these issues, she considered the activity usable, stating, "I think it could definitely be [more] smooth, but I could use it tomorrow." She also indicated that she would like to use this type of design frequently.

**Case Study 2:** P2 is a high school French teacher, teaching beginner French through advanced French. She spent $20$ minutes authoring a conversation practice activity for one of her French 1 students using vocabulary and phrases from Unit 1 of their textbook. P2 began by uploading screenshots of the Unit 1 vocabulary list as reference material. The resulting interaction simulated a first meeting in French using beginner-level vocabulary, with gentle modeling, French-only reminders, and vocabulary support when needed.

During interaction preview, P2 noticed that the robot used the phrase "C'est fantastique," which she did not want her students to learn. She returned to the activity specification and instructed RIDE, "Please don't say C'est fantastique, say instead c'est superbe" This change reflected her domain-specific judgment about the specific language preferences she has for her students. In the post-study interview, she also said she would want RIDE to remember such preferences

Table 1: Participant demographics. Edu. stands for education.

| P | Age | Gender | English Proficiency | Edu. | Field of Study | Profession | Programming Experience | Familiarity with AI | Preferred Device |
|---|---|---|---|---|---|---|---|---|---|
| P1 | 27 | Female | Native | M.S. | Speech language pathology & autism studies | Speech language therapist | None | Familiar | Laptop |
| P2 | 54 | Female | Advanced | Ph.D. | French literature | High school French teacher | None | Familiar | Laptop |
| P3 | 60 | Female | Advanced | B.S. | Nursing | Registered nurse | None | Some experience | Phone |

for future activities rather than requiring her to specify them again each time.

P2 enjoyed the overall design process ($4/5$), but encountered several problems with French speech processing during both authoring and on-robot testing. She rated RIDE's usability at 67.5 on the SUS and her satisfaction with the resulting robot activity at $3.67/5$. During authoring, the voice agent failed twice to recognize the French name "Pierre." When P2 asked for help, the researcher suggested that P2 enter the name using text instead, which resolved the issue. During the first on-robot test, the robot also remained set to English because P2 had missed the language setting in the configuration interface, requiring a reminder from the researcher to change it to French.

Speech-processing errors continued during on-robot testing. The speech-to-text service frequently mistranscribed P2's French responses, and the text-to-speech service pronounced two French words using English pronunciation. P2 pointed out that parts of the interaction "was very smooth" and "[the robot] did exactly what he was suppose to do," but some portions of the conversation "felt stagnant." Hence, the researcher reviewed the system-generated conversation transcript with P2, revealing that the conversational breakdowns were all caused by speech-to-text errors. The LLM automatically recovered from some mistranscriptions, while others led the robot to provide unnecessary corrective guidance and disrupted the flow of the conversation.

After reviewing the transcript, P2 distinguished these speech-processing failures from the activity design itself. She noted that "[the robot] is good when he understands me. It's not his fault." Despite the breakdowns, she could still imagine using RIDE and the robot in her classroom, explaining that "these are more like... small obstacles, little obstacles, but I see the bigger picture, I'm actually very much looking forward to it... because this is fun."

**Case Study 3:** P3 works as a registered nurse and designed a stress-relief activity for herself. Unlike the two domain experts, she did not begin with a specific interaction in mind. She spent $15$ minutes authoring on her phone and chose text rather than speech to communicate with RIDE. After manually completing the target user profile, she developed a satisfactory activity using only 21 words during the activity specification phase (see full design conversation in Fig. 4). P3 attributed this low effort in part to the form of RIDE's questions, explaining that they "just give me options of what I will type, so it's easier for me because, yeah, I don't want to think anymore." The resulting interaction guided her through a structured calming session involving breathing exercises, body awareness, guided visualization, and a mindfulness check-in.

P3 enjoyed the design process ($4/5$) and reported high satisfaction with the resulting activity ($4.83/5$). She also described the interaction with RIDE as straightforward, stating that "nothing [was] challenging" and that "everything was easy to understand and go through." However, she gave RIDE a SUS score of $57.5$. Her interview comments also suggested that, despite finding the system easy to use, she was uncertain whether she would use this type of technology regularly. As she explained, "I'm not that kind of person who will do it... I'm not into this kind of advanced technology... if I'm younger, for sure I will do it."

# Discussion

## Division of Labor for Rapid Activity Authoring

Across the three case studies, no participant wrote a prompt, assembled a program, or edited a state-level prompt by hand, and none had prior programming experience. Instead, participants described the activity they wanted through conversation, while RIDE handled decomposition into states, sequencing, transition logic, state-level prompt construction, and deployment to the robot. All three participants reached a design they were willing to deploy within seven conversational turns and were generally satisfied with the resulting activity after testing it on the robot.

RIDE's clarifying questions appeared to shift some of the authoring work from implementation to decision-making about the interaction itself. P1 and P2 both reported that the questions were "really helpful" and helped them think during activity specification. These questions prompted participants to consider aspects of the activity they had not yet specified, such as how the interaction should end or how the robot should respond when the user struggled. Participants, therefore, did not need to determine how to implement the interaction, but they still made consequential decisions about what the interaction should be.

Reference materials provided another low-effort way to contribute domain-specific information. Both domain experts uploaded materials from their existing practice and felt that the resulting activities reflected them. P1 uploaded a lesson plan for the therapy session in which the activity would be used, while P2 uploaded screenshots of a textbook vocabulary list. Rather than translating this material into prompts or program logic, participants could supply it directly as context for the design.

Participants' other contributions were often brief. P3 began without a specific activity in mind and produced a stress-

relief activity using only 21 words in total. She attributed this to the form of the agent's questions, which "just give me options of what I will type, so it's easier for me because, yeah, I don't want to think anymore." Offering concrete options can therefore support users who have not yet formed a complete specification by reducing the effort required to articulate one. Participants did not, however, delegate all aspects of the design equally. They readily handed off interaction structure while retaining control over domain-relevant details such as target phrases, vocabulary, and content to avoid. **This suggests a useful division of labor for low-effort authoring: the system can propose structure and candidate content, while users supply, select, and verify the details that encode their goals and domain knowledge.** Preserving this user involvement is important because excessive off-loading to LLMs may allow unexpected errors to go unnoticed. Future systems should intentionally design mechanisms to support active user review and approval, particularly before deploying generated interactions with sensitive users such as children.

This division of labor may also extend beyond interaction authoring to robot configuration. RIDE currently requires users to select robot configurations separately from the activity design. During P2's on-robot test, for example, the robot remained configured for English even though she had designed French conversation practice for beginning students and uploaded a French vocabulary list. The information needed to infer the appropriate language was already present in the activity specification and target user profile, but the configuration remained disconnected from that context. Parameters such as language, facial expression, and body movement could instead be inferred from the activity and surfaced for confirmation, reducing another source of authoring effort. However, such automation should remain inspectable and editable, because configuration choices can themselves carry domain-specific meaning, *e.g.,* when a therapist deliberately selects an expression to support a social-skills goal.

## Limitations & Future Work

Our evaluation involved three participants in single sessions. Two of the activities were designed for students or patients who did not take part in the study. Participants therefore evaluated breakdowns on their own behalf. Intended end users may respond differently. For example, a student who is repeatedly misunderstood by the robot or a child who becomes confused during an activity may be less tolerant of the same glitches. Future work should evaluate RIDE with both authors and intended end users. It should also examine whether authors revise activities differently after observing them in use. The study also did not compare RIDE with existing authoring approaches, such as visual or block-based robot programming tools. Future work should conduct controlled comparative studies. These studies could examine differences in authoring effort, usability, design quality, and user control. Longer-term studies with a broader range of participants are also needed to understand how authoring needs evolve over repeated use.

System responsiveness was another limitation. P1 and P2 both wanted shorter response times from the RIDE voice agent, which took 46.54 seconds on average before producing its first spoken word. This latency could be reduced by decoupling elicitation from generation, allowing the agent to continue asking clarifying questions while the activity prompt and interaction flow are generated. Participants also suggested making agent guidance more concise and providing contextual tips, such as recommending text input when speech recognition is unreliable.

Speech processing further constrained non-English use. In Case Study 2, speech-to-text frequently mistranscribed French input, and text-to-speech applied English pronunciation to two French words. Because language learning is one of RIDE's target applications, stronger multilingual speech support is an important direction for future work.

RIDE also does not yet learn an author's preferences across activities. Although it retains target user profiles and allows prior activities to be duplicated, each new design otherwise begins from the same starting point. Participants wanted the system to carry forward recurring preferences about activity structure, content to include, and content to avoid. Maintaining such preferences across sessions could further reduce the effort required for repeated authoring. Participants also suggested more direct revision mechanisms, such as annotating the interaction preview and having RIDE incorporate those edits automatically.

Finally, RIDE does not provide a built-in robot simulation. We made this choice to remain agnostic to robot platforms and to encourage testing on the physical robot, but it limits users' ability to preview embodiment-specific behavior before deployment. Future work could explore lightweight preview mechanisms that preserve platform independence while providing more information about how an interaction will appear on the robot. RIDE was implemented and evaluated with the open-source M robot (Antony et al. 2026b); although we provide the web application, ROS integration, and porting instructions as open source, further work is needed to evaluate how well the approach transfers across robot platforms.

## Acknowledgments

This work was supported by National Science Foundation awards #2143704 and #2141335. The work was also supported by the National Science Foundation Research Experiences for Undergraduates (REU) program.

**AI Statement.** Generative AI was used for proofreading, copyediting, and creating initial table and figure mock-ups in this manuscript. AI was also used during system development. All resulting content was edited, reviewed, and verified by the authors.

**CRediT author statement.** Shiye Cao: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing, Visualization, Project Administration. Aureliane Tchamdja: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing. Chien-Ming Huang: Conceptualization, Methodology, Writing-Review and editing, Supervision, Funding acquisition.

## References


Ajaykumar, G.; Steele, M.; and Huang, C.-M. 2021. A survey on end-user robot programming. *ACM Computing Surveys (CSUR)*, 54(8): 1–36.

Akgun, B.; Cakmak, M.; Yoo, J. W.; and Thomaz, A. L. 2012. Trajectories and keyframes for kinesthetic teaching: A human-robot interaction perspective. In *Proceedings of the seventh annual ACM/IEEE international conference on Human-Robot Interaction*, 391–398.

Alexandrova, S.; Tatlock, Z.; and Cakmak, M. 2015. RoboFlow: A flow-based visual programming language for mobile manipulation tasks. In *2015 IEEE international conference on robotics and automation (ICRA)*, 5537–5544. IEEE.

Antony, V. N.; Cao, S.; Wang, S.; and Huang, C.-M. 2026a. ELLA: Generative AI-powered social robots for early language development at home. In *Proceedings of the 25th Annual ACM Interaction Design and Children Conference*, 173–193.

Antony, V. N.; Gong, Z.; Kim, Y.; and Huang, C.-M. 2026b. Introducing M: A Modular, Modifiable Social Robot. *arXiv preprint arXiv:2603.19134*.

Antony, V. N.; Stiber, M.; and Huang, C.-M. 2025. Xpress: A system for dynamic, context-aware robot facial expressions using language models. In *2025 20th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, 958–967. IEEE.

Bangor, A.; Kortum, P.; and Miller, J. 2009. Determining what individual SUS scores mean: Adding an adjective rating scale. *Journal of usability studies*, 4(3): 114–123.

Brooke, J. 1996. Sus: a "quick and dirty'usability. *Usability evaluation in industry*, 189(3): 189–194.

Buchina, N.; Kamel, S.; and Barakova, E. 2016. Design and evaluation of an end-user friendly tool for robot programming. In *2016 25th IEEE International symposium on robot and human interactive communication (RO-MAN)*, 185–191. IEEE.

Cao, S.; Moon, J.; Mahmood, A.; Antony, V. N.; Xiao, Z.; Liu, A.; and Huang, C.-M. 2025. Interruption handling for conversational robots. *2025 Robotics: Science and Systems*.

Cao, S.; Moon, J.; Xu, Y.; Liu, A.; and Huang, C.-M. 2026. Reframing Conversational Design in HRI: Deliberate Design with AI Scaffolds. In *Proceedings of the 21st ACM/IEEE International Conference on Human-Robot Interaction*, 59–67.

Coronado, E.; Mastrogiovanni, F.; Indurkhya, B.; and Venture, G. 2020. Visual programming environments for end-user development of intelligent and social robots, a systematic review. *Journal of Computer Languages*, 58: 100970.

Csapo, A.; Gilmartin, E.; Grizou, J.; Han, J.; Meena, R.; Anastasiou, D.; Jokinen, K.; and Wilcock, G. 2012. Multimodal conversational interaction with a humanoid robot. In *2012 IEEE 3rd International Conference on Cognitive Infocommunications (CogInfoCom)*, 667–672. IEEE.

Gao, Y.; and Huang, C.-M. 2019. PATI: a projection-based augmented table-top interface for robot programming. In *Proceedings of the 24th international conference on intelligent user interfaces*, 345–355.

Ikeda, B.; and Szafir, D. 2024. Programar: Augmented reality end-user robot programming. *ACM Transactions on Human-Robot Interaction*, 13(1): 1–20.

Jain, S.; Thiagarajan, B.; Shi, Z.; Clabaugh, C.; and Matarić, M. J. 2020. Modeling engagement in long-term, in-home socially assistive robot interventions for children with autism spectrum disorders. *Science robotics*, 5(39): eaaz3791.

Kapinus, M.; Beran, V.; Materna, Z.; and Bambušek, D. 2019. Spatially situated end-user robot programming in augmented reality. In *2019 28th IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)*, 1–8. IEEE.

Matheus, K.; Ramnauth, R.; Scassellati, B.; and Salomons, N. 2025. Long-term interactions with social robots: Trends, insights, and recommendations. *ACM Transactions on Human-Robot Interaction*, 14(3): 1–42.

Oppenlaender, J.; Linder, R.; and Silvennoinen, J. 2025. Prompting AI art: An investigation into the creative skill of prompt engineering. *International journal of human–computer interaction*, 41(16): 10207–10229.

Paxton, C.; Hundt, A.; Jonathan, F.; Guerin, K.; and Hager, G. D. 2017. CoSTAR: Instructing collaborative robots with behavior trees and vision. In *2017 IEEE international conference on robotics and automation (ICRA)*, 564–571. IEEE.

Porfirio, D.; Fisher, E.; Sauppé, A.; Albarghouthi, A.; and Mutlu, B. 2019. Bodystorming human-robot interactions. In *proceedings of the 32nd annual ACM symposium on user Interface software and technology*, 479–491.

Porfirio, D. J.; Stegner, L.; Cakmak, M.; Sauppé, A.; Albarghouthi, A.; and Mutlu, B. 2021. Figaro: A tabletop authoring environment for human-robot interaction. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 1–15.

Pot, E.; Monceaux, J.; Gelin, R.; and Maisonnier, B. 2009. Choregraphe: a graphical tool for humanoid robot programming. In *Ro-man 2009-the 18th ieee international symposium on robot and human interactive communication*, 46–51. IEEE.

Sanders, D. A.; Gegov, A.; and Ndzi, D. 2016. Knowledge-based expert system using a set of rules to assist a teleoperated mobile robot. In *Proceedings of SAI Intelligent Systems Conference*, 371–392. Springer.

Sato, A. J.; Kim, C. Y.; White, N. T.; Maneesh, A.; Wang, Y.; Ho, H.-R.; and Mutlu, B. 2026. Robo-Blocks: Generative Scaffolding in End-User Design and Programming of Social Robots. In *Proceedings of the 2026 Designing Interactive Systems Conference*, 3763–3779.

Schoen, A.; and Mutlu, B. 2024. OpenVP: A customizable visual programming environment for robotics applications. In *Proceedings of the 2024 ACM/IEEE International Conference on Human-Robot Interaction*, 944–948.

Schoen, A.; White, N.; Henrichs, C.; Siebert-Evenstone, A.; Shaffer, D.; and Mutlu, B. 2022. CoFrame: A system for training novice cobot programmers. In *2022 17th ACM/IEEE In-*

*ternational Conference on Human-Robot Interaction (HRI)*, 185–194. IEEE.

Sefidgar, Y. S.; Weng, T.; Harvey, H.; Elliott, S.; and Cakmak, M. 2018. Robotist: Interactive situated tangible robot programming. In *Proceedings of the 2018 ACM Symposium on Spatial User Interaction*, 141–149.

Swaminathan, M.; Hsu, L.-J.; Thant, M. M.; Amon, K. J.; Kim, A. S.; Tsui, K. M.; Sabanović, S.; Crandall, D. J.; and Khoo, W. 2024. If [yourname] can code, so can you! end-user robot programming for non-experts. In *Companion of the 2024 ACM/IEEE International Conference on Human-Robot Interaction*, 1033–1037.

Weintrop, D.; Afzal, A.; Salac, J.; Francis, P.; Li, B.; Shepherd, D. C.; and Franklin, D. 2018. Evaluating CoBlox: A comparative study of robotics programming environments for adult novices. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 1–12.

Weintrop, D.; Shepherd, D. C.; Francis, P.; and Franklin, D. 2017. Blockly goes to work: Block-based programming for industrial robots. In *2017 IEEE Blocks And Beyond Workshop (B&B)*, 29–36. IEEE.

Weintrop, D.; and Wilensky, U. 2017. Comparing block-based and text-based programming in high school computer science classrooms. *ACM Transactions on Computing Education (TOCE)*, 18(1): 1–25.

Wu, T.; Jiang, E.; Donsbach, A.; Gray, J.; Molina, A.; Terry, M.; and Cai, C. J. 2022. Promptchainer: Chaining large language model prompts through visual programming. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, 1–10.

Wu, T.; Terry, M.; and Cai, C. J. 2022. Ai chains: Transparent and controllable human-ai interaction by chaining large language model prompts. In *Proceedings of the 2022 CHI conference on human factors in computing systems*, 1–22.

Yigitbas, E.; Jovanovikj, I.; and Engels, G. 2021. Simplifying robot programming using augmented reality and end-user development. In *IFIP Conference on Human-Computer Interaction*, 631–651. Springer.

Zamfirescu-Pereira, J. D.; Wong, R. Y.; Hartmann, B.; and Yang, Q. 2023. Why Johnny can't prompt: how non-AI experts try (and fail) to design LLM prompts. In *Proceedings of the 2023 CHI conference on human factors in computing systems*, 1–21.

Zhang, J.; and Arawjo, I. 2025. Chainbuddy: An ai-assisted agent system for generating llm pipelines. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, 1–21.